

Guía de Defensa

Papaws

Sistema de microservicios distribuidos (.NET + Cassandra + React)

Cómo usar esta guía

Esta guía está en formato **pregunta** → **respuesta**. Cada tema tiene primero una explicación clara del concepto y después las preguntas que te pueden hacer, con la respuesta lista para decir y el lugar exacto del código donde se ve. Búscala por sección o por palabra clave.

Estructura de cada bloque:

- **P:** la pregunta probable del docente.
- **Respuesta:** lo que conviene responder, explicado en simple.
- **Dónde mirar:** archivo y método para mostrarlo en vivo si lo piden.

Precisión importante sobre el despliegue. Donde antes esta guía hablaba de «una base de datos por servicio», lo correcto es «un *keyspace* por servicio». Los dos *keyspaces* conviven en un **único clúster Cassandra de dos nodos** con `replication_factor = 2`: la separación entre servicios es **lógica** (cada servicio abre solo su *keyspace*), no física. Esto se detalla en las secciones [13](#) y [15](#), y *refuerza* la defensa sobre tolerancia a fallos.

Índice

Cómo usar esta guía	1
1. La arquitectura en una frase	3
2. Los tres flujos clave (quién llama a quién)	3
2.1. ¿Dónde llaman los reportes a los rescatistas? (C1 «rescatista por id» / C2 «animales por rescatista»)	3
2.2. ¿Dónde llaman los rescatistas a los animales? (clic en «Ver ficha»)	4
2.3. ¿Dónde llaman los animales a sus consultas?	4
3. Modelado en Cassandra (query-first)	5
4. Trazabilidad e historial (eventos inmutables)	5
5. Integridad entre servicios (sin foreign keys)	5
6. Concurrencia: el stock de productos (LWT)	6
7. Gastos y precios (refugio sin fines de lucro)	6
8. Refugio y cupo de rescatistas	7
9. Seguridad: JWT y roles	7

10. Reglas de negocio	7
11. Paginación	8
12. Reportes (el servicio agregador)	8
12.1. Catálogo de reportes: qué significa cada código Cn	8
12.2. Cómo se arma un reporte	9
13. Ver las queries reales en Cassandra	9
13.1. Forma A — Una query puntual (cq sh)	9
13.2. Forma B — Capturar TODAS las queries (la que más sirve)	9
13.3. Forma C — Inspeccionar datos directos (demostrar la desnormalización)	10
13.4. Forma D — Logs	10
14. Comandos de terminal para la demo	10
14.1. Ver que todo está corriendo	10
14.2. Demostrar que es un clúster distribuido (2 nodos)	11
14.3. Ver los keyspaces y su factor de réplica	11
14.4. Ver las tablas y sus claves (modelado query-first)	11
14.5. Contar y ver datos de una tabla	11
14.6. Levantar, reiniciar y ver logs	11
14.7. Front en modo desarrollo	11
14.8. Probar un endpoint por terminal (con token)	11
14.9. Chuleta: «si te pide esto, escribís esto»	12
14.10. Paso a paso: mostrar una consulta en Cassandra (desde DESCRIBE)	13
15. Partes del código fáciles de cambiar (en vivo)	14
15.1. Cupo máximo de animales por rescatista	14
15.2. Cuántos elementos por página	14
15.3. Precios de servicios y productos sembrados	14
15.4. Símbolo y formato de la moneda	14
15.5. Tamaño de página por defecto del backend	15
15.6. Umbral de «stock bajo»	15
15.7. Catálogos: especies, especialidades, tipos de organización	15
15.8. Una regla de validación	15
16. Resumen: quién llama a quién	15
17. Preguntas trampa y cómo salir bien	15
18. Banco de preguntas rápidas	16
18.1. Arquitectura y comunicación	16
18.2. Cassandra	16
18.3. .NET y front	17
18.4. Negocio	17

1. La arquitectura en una frase

Papaws son **TRES microservicios .NET independientes** más un frontend React que los consume. Dos servicios tienen **keyspace propio** de Cassandra; el tercero (Reportes) **no tiene datos propios**. Ningún servicio entra al keyspace de otro: si necesita datos ajenos, los pide por la API REST.

- **Animales** (puerto 8080): rescatistas, organizaciones y animales. Es el dueño de esos datos. También hace el **login** y emite el **JWT**.
- **Consulta** (puerto 8081): veterinarios, servicios, productos (inventario) y consultas clínicas.
- **Reportes** (puerto 8082): **NO** tiene datos propios. Es un **AGREGADOR**: arma los reportes pidiendo datos por HTTP a los otros dos.
- **Front** (Pawpaws-Front, React + Vite): decide a qué servicio ir por una «zona» (animales / consulta) o por la base de reportes.

P: ¿Por qué microservicios y no un monolito?

Para desacoplar dominios: cada servicio se despliega, escala y falla por separado. El precio a pagar es que no hay JOINS ni transacciones entre servicios; la consistencia se cuida a mano al escribir (sección 5). Es la decisión central a defender.

P: ¿Cómo sabe el front a qué microservicio pegar?

Cada llamada declara su «zona». La función `baseFor()` traduce la zona a la URL del servicio. Los endpoints concretos (qué ruta, qué método) están centralizados en un solo archivo. Reportes no es una «zona»: va por su propia constante (`REPORTES_BASE`) y una función aparte.

Dónde mirar: `Pawpaws-Front/src/api/client.ts` (`baseFor` / `apiGet` / `apiPost`) y `Pawpaws-Front/src/api/endpoints.ts`

P: ¿La regla de oro de la arquitectura?

Un servicio **NUNCA** lee ni escribe el keyspace de otro. Toda dependencia cruzada pasa por la API REST del dueño, reenviando el token JWT del usuario.

2. Los tres flujos clave (quién llama a quién)

Estas son las tres preguntas de recorrido «de punta a punta» más probables. Conviene poder trazarlas: **front** → **controlador** → **servicio** → **tabla de Cassandra**.

¿Qué son «C1», «C2», «C20»? Son los **códigos de los reportes** (las consultas de lectura del sistema). El proyecto se diseñó *query-first*: primero se listaron todas las preguntas que el sistema debe poder responder y se numeraron C1, C2, C3... (del diagrama de queries, estilo «Chebotko» de Cassandra). Cada **Cn** es una **consulta concreta**; por ejemplo **C2** = «animales por rescatista» y **C20** = «rescatistas y animales por organización». La tabla con el significado de cada código está en la sección 12.

2.1 ¿Dónde llaman los reportes a los rescatistas? (C1 «rescatista por id» / C2 «animales por rescatista»)

Recorrido:

- **Front**: en `Reportes.tsx` se elige el reporte y se llama `reportesApi.c1_rescatistaPorId(id)` o `c2_animalesPorRescatista(id)`. Pegan al servicio de **Reportes**, no a Animales.
- **Reportes**: entra por `RescatistasReportesController.cs` y llama a `ReporteService` (`C1_RescatistaPorIdAsync` / `C2_AnimalesPorRescatistaAsync`).

- Aquí «los reportes llaman a los rescatistas»: ReporteService no tiene base; usa el cliente HTTP AnimalesClient.GetRescatistaByIdAsync(id) → GET api/rescatistas/{id} contra Animales. Para C2 además GetAnimalesByRescatistaAsync(id) → GET api/animales/rescatista/{id}.
- Animales responde desde RescatistasController → RescatistaService, leyendo la tabla rescatis-tas_by_id.

Dónde mirar: Pawpaws-Reportes/Services/AnimalesClient.cs y ReporteService.cs ; Pawpaws-Animales/Services/RescatistaService.cs

P: ¿Y la autenticación no se pierde al saltar de servicio?

No. El cliente de Reportes reenvía el token JWT del usuario (copia el header Authorization), así Animales sigue exigiendo login y rol. Ese reenvío está en el método que arma el request del cliente.

Dónde mirar: Pawpaws-Reportes/Services/AnimalesClient.cs (CrearRequest copia Authorization)

2.2 ¿Dónde llaman los rescatistas a los animales? (clic en «Ver ficha»)

- Front: el botón «Ver ficha» en Rescatistas.tsx navega a /rescatistas/:id (ruta en App.tsx) y la pinta RescatistaDetalle.tsx.
- Aquí «el rescatista llama a sus animales»: RescatistaDetalle.tsx hace animalesApi.porRescatista(id) → GET /api/animales/rescatista/{id} contra Animales.
- Animales: AnimalesController.ObtenerPorRescatista → AnimalService.ObtenerPorRescatistaAsync, que lee la tabla animales_by_rescatista.

Dónde mirar: Pawpaws-Front/src/pages/RescatistaDetalle.tsx ; Pawpaws-Animales/Services/AnimalService.cs

P: ¿Por qué existe una tabla animales_by_rescatista aparte de animales_by_id?

Está desnormalizada y particionada por rescatista_id justo para que «dame todos los animales de este rescatista» sea UNA sola lectura de una partición, sin JOIN ni scan. El mismo animal se guarda también en animales_by_id para buscarlo por su propio id. Ambas se escriben juntas en un BatchStatement para que no se desincronicen.

Dónde mirar: Pawpaws-Animales/Data/CassandraSchema.cs y GuardarAnimalAsync en AnimalService.cs

2.3 ¿Dónde llaman los animales a sus consultas?

- Front: AnimalDetalle.tsx hace consultasApi.porAnimal(id) → GET /api/consultas/animal/{id} contra el servicio de Consulta (las consultas viven en otro microservicio).
- Consulta: ConsultasController.ObtenerPorAnimal → ConsultaService.ObtenerPorAnimalAsync.
- En Cassandra: primero lee la tabla índice consulta_codigos_by_animal (particionada por animal_id) para sacar los códigos; luego trae cada consulta de consultas_by_codigo. Es el patrón «tabla índice + lectura por clave».

Dónde mirar: Pawpaws-Front/src/pages/AnimalDetalle.tsx ; Pawpaws-Consulta/Services/ConsultaService.cs

P: Si el animal y la consulta están en servicios distintos, ¿cómo se relacionan sin foreign key?

La consulta guarda el animal_id. No hay FK que cruce servicios; la relación se sostiene por ese id y la consistencia se cuida al CREAR la consulta, validando el animal por HTTP (sección 5).

3. Modelado en Cassandra (query-first)

Cassandra no hace JOINS ni filtra eficiente por columnas que no sean la clave de partición. Por eso se modela «una tabla por consulta de lectura» (*query-first*) y el mismo dato se guarda varias veces (*desnormalización*). La clave de partición define dónde vive el dato y por qué campo se puede leer rápido.

Ejemplos en el sistema:

- **Animales:** animales_by_id (por id) y animales_by_rescatista (por rescatista).
- **Consultas:** consultas_by_codigo (por código); consulta_codigos_by_animal y consulta_codigos_by_veterinario (índices para listar); consulta_servicios_by_codigo y consulta_productos_by_codigo (relaciones de la consulta).

Dónde mirar: `Pawpaws-Animales/Data/CassandraSchema.cs` y `Pawpaws-Consulta/Data/CassandraSchema.cs`

P: ¿Y si quiero saber en qué consultas se usó un servicio? (consulta inversa)

No se puede con consulta_servicios_by_codigo, porque esa tabla está particionada por código (solo responde «qué servicios tiene ESTA consulta»). Por eso se agregó una tabla de **índice inverso**, consulta_codigos_by_servicio, particionada por servicio_id, que se mantiene sincronizada al crear, editar y borrar una consulta. La usa la ficha de servicio.

Dónde mirar: `ObtenerPorServicioAsync` y los `INSERT/DELETE` del índice en `Pawpaws-Consulta/Services/ConsultaService.cs`

P: ¿No es peligroso duplicar datos?

El riesgo es la desincronización. Se controla escribiendo TODAS las copias en la misma operación (`BatchStatement`) para que entren juntas, y manteniendo los índices en cada alta/baja/edición.

4. Trazabilidad e historial (eventos inmutables)

Para no perder historia, los cambios se guardan como **eventos inmutables** en tablas ordenadas por fecha descendente (*clustering*). En vez de pisar un valor, se agrega un evento nuevo.

- eventos_custodia_by_animal: ingreso del animal y cada reasignación entre rescatistas.
- eventos_organizacion_by_rescatista: alta y cambios de organización del rescatista.
- eventos_adopcion_by_animal: adopciones y devoluciones.

Dónde mirar: *Tablas y backfill en* `Pawpaws-Animales/Data/CassandraSchema.cs`; *registro en* `AnimalService.cs` y `RescatistaService.cs`; *líneas de tiempo en* `AnimalDetalle.tsx` y `RescatistaDetalle.tsx`

P: ¿Por qué el evento guarda el nombre y no solo el id?

Para que el historial siga siendo legible aunque después se dé de baja ese rescatista u organización. Si guardara solo el id, al borrarse quedaría un id muerto sin sentido. Es un «*snapshot*» del nombre en el momento del evento.

P: ¿Qué es el backfill?

Un proceso **idempotente** que siembra los eventos iniciales para los datos que ya existían antes de tener el historial, sin duplicar si se corre dos veces.

5. Integridad entre servicios (sin foreign keys)

Como cada servicio tiene su keyspace, no hay claves foráneas que crucen servicios. La consistencia se cuida al **ESCRIBIR**:

- Al crear una consulta, Consulta valida el animal **llamando por HTTP** a Animales (con reintentos). Si el animal no existe, rechaza.
- Esa misma validación trae el **estado** del animal y rechaza agendar si está «Adoptado» (un adoptado no recibe consultas hasta que se registre su devolución).

Dónde mirar: `Pawpaws-Consulta/Services/AnimalReferenceService.cs` (`ObtenerEstadoAsync`) usado en `CrearAsync` de `ConsultaService.cs`

P: ¿Y si el servicio de Animales está caído cuando creo la consulta?

El cliente reintenta con *backoff*. Si aun así no hay respuesta, la creación **FALLA** con un error controlado, en lugar de crear una consulta que apunte a un animal que no se pudo verificar. Se prefiere fallar a dejar datos inconsistentes.

P: ¿Cómo se borra un animal sin dejar consultas huérfanas?

El animal se borra **físicamente** y, en cascada, se borran sus consultas. Si la baja de la cascada falla, se propaga el error y el animal **NO** se borra, para no dejar consultas apuntando a un animal inexistente.

Dónde mirar: `EliminarAsync` en `Pawpaws-Animales/Services/AnimalService.cs` (*cascada cross-servicio*)

6. Concurrencia: el stock de productos (LWT)

Si dos consultas descuentan stock del mismo producto a la vez, un UPDATE normal podría pisar el valor del otro (*lost update*). Se resolvió con **compare-and-set** usando **LWT** (transacciones ligeras) de Cassandra: `UPDATE ... IF stock_disponible = ?`. Si otro proceso cambió el stock entremedio, el IF falla, se lee el valor real que devuelve Cassandra y se reintenta.

Dónde mirar: `AjustarStockAsync` en `Pawpaws-Consulta/Services/ProductoService.cs`

P: ¿Por qué no usar solo un UPDATE común?

Porque dos UPDATE concurrentes no se enteran uno del otro: el último gana y se pierde un descuento. El LWT agrega una condición (IF) que solo aplica si el valor sigue siendo el que leí; si cambió, reintento con el valor nuevo. Es control de concurrencia **optimista**.

P: ¿Es caro el LWT?

Sí, más que un *write* normal porque usa consenso (Paxos) entre réplicas. Se justifica solo donde hay riesgo real de carrera, como el stock. No se usa para todo.

7. Gastos y precios (refugio sin fines de lucro)

El reporte de gastos lo calcula **el servicio de Consulta**, porque ahí viven los precios: `precio_base` de los servicios y `costo_unitario` de los productos. Por cada consulta suma el costo de sus servicios más el de los productos consumidos (`costo_unitario × cantidad`). El front lo agrupa por mes.

- **Endpoint:** `GET /api/consultas/gastos` (`ObtenerGastosAsync` en `ConsultaService.cs`).
- Las consultas **canceladas** se **EXCLUYEN**: devuelven el stock y no representan gasto real.
- Los precios son en bolivianos (Bs) y a precio de costo, porque es un refugio sin fines de lucro: no hay margen comercial.

Dónde mirar: `Pawpaws-Consulta/Services/ConsultaService.cs` (`ObtenerGastosAsync`); `Pawpaws-Front/src/pages/Gastos.tsx`

P: ¿Por qué el cálculo de gastos no está en Reportes?

Porque todos los datos necesarios (precio de servicios y costo de productos) viven en Consulta. Calcularlo ahí evita que Reportes tenga que pedir y recombinar todo por HTTP: se hace donde están los datos.

P: ¿Por qué los números se muestran sin punto de miles?

Decisión de formato: el punto se reserva para decimales. La moneda se muestra como «Bs» con el locale es-BO y agrupación desactivada, para que no se confunda 1.000 (mil) con 1,000 (uno coma cero).

8. Refugio y cupo de rescatistas

Existe un rescatista interno llamado «**Refugio**» (oculto de la gestión) que actúa como destino institucional de los animales. Cada rescatista normal tiene un **cupo máximo** de animales; el Refugio no tiene límite.

- Al dar de baja un rescatista, sus animales se reasignan: llenan el cupo del rescatista elegido y lo que sobra se deriva automáticamente al Refugio.
- Al reasignar un animal suelto, se bloquea si el rescatista destino ya está en su cupo. Así se reparten los animales del Refugio sin sobrecargar a nadie.

Dónde mirar: `ReasignarAnimalesAsync` y la validación de cupo en `ActualizarAsync` de `Pawpaws-Animales/Services/AnimalService.cs`; constante `CapacidadMaxima` en `Models/Rescatista.cs`

P: ¿Por qué no asignar todos los animales a un solo rescatista al dar de baja?

Porque un rescatista no puede hacerse cargo de más animales de los que su cupo permite. El sistema respeta ese límite y manda el excedente al Refugio, desde donde luego se reparten de a poco a quienes tengan lugar.

9. Seguridad: JWT y roles

Login con **JWT** y autorización por **roles** (Administrador, Encargado de consultas, Encargado de rescatistas). Cada endpoint se protege con `[Authorize(Roles = ...)]`. Cuando un servicio llama a otro, **reenvía** el token del usuario, así la autorización se respeta en toda la cadena.

- El secreto de firma del JWT y la contraseña NO están en el repositorio: se leen de variables de entorno / `user-secrets`, y `appsettings.json` queda en blanco (*fail-fast* si faltan).
- La misma clave de firma es idéntica en los tres servicios: Animales firma, los demás validan.

Dónde mirar: Carpetas `Security/` de cada servicio; emisión del token en `Pawpaws-Animales (AuthController / JwtService)`

P: ¿Cómo respeta Reportes los permisos si no tiene los usuarios?

No necesita la base de usuarios: confía en el JWT firmado. Valida la firma con la clave compartida y lee los roles del token. Además reenvía ese token a `Animales/Consulta`, que vuelven a validar.

10. Reglas de negocio

Ejemplos concretos que conviene tener a mano:

- Una consulta no puede nacer «Completada».
- Un animal «Adoptado» solo puede pasar a «Devuelto al refugio», no directo a «Disponible».
- No se permiten productos (ni organizaciones) con nombre repetido entre los activos.
- La fecha de ingreso de un animal no puede ser futura.
- Un rescatista no puede superar su cupo de animales (sección 5 y «Refugio y cupo»).

P: ¿Borrado físico o lógico?

Rescatistas, veterinarios, servicios, productos y organizaciones usan **borrado lógico** (`activo = false`) para no romper referencias históricas. Los animales SÍ se borran físicamente, y al hacerlo se borran en cascada sus consultas.

11. Paginación

Hay paginación en **dos niveles**. En el **backend**, los listados aceptan `?pagina=` y `?tamano=` y devuelven un objeto con los ítems de esa página. En el **front**, además, las listas largas se paginan del lado del cliente: 10 elementos por página con controles numerados (1, 2, 3...).

- **Backend:** helper `Paginar()` y un tamaño por defecto; el front pide la primera página con tamaño alto.
- **Front:** hook `usePaginated` y componente `Pagination`, aplicados a animales, productos, rescatistas, servicios, veterinarios, organizaciones, consultas y gastos.

Dónde mirar: `apiList` en `Pawpaws-Front/src/api/client.ts`; `Pawpaws-Front/src/hooks/usePaginated.ts` y `components/Pagination.tsx`; `Common/Paginacion.cs` en cada servicio

P: ¿Por qué paginar también en el front si el backend ya pagina?

Porque el front trae la lista y la filtra/ordena en memoria para una UX fluida; paginar ahí evita renderizar listas gigantes de cientos de tarjetas. Son dos optimizaciones que se complementan.

12. Reportes (el servicio agregador)

12.1 Catálogo de reportes: qué significa cada código C_n

Cada reporte es una **consulta de lectura** con un código C_n (sección 2). No se trata de tablas ni de código interno: es el **nombre de la pregunta** que el reporte responde. Esta es la lista completa:

Código	Qué pregunta responde	Sobre
C1	Rescatista por ID	Rescatistas
C2	Animales por rescatista	Rescatistas
C19	Rescatistas por zona	Rescatistas
C20	Rescatistas y animales por organización	Rescatistas
C21	Rescatistas por tipo de organización	Rescatistas
C3	Animales por especie	Animales
C4	Consultas por animal	Animales
C17	Animales por nombre	Animales
C15	Consulta por código	Consultas
C6	Detalle de una consulta	Consultas
C7	Consultas por estado	Consultas
C8	Servicios usados en una consulta	Consultas
C9	Productos usados en una consulta	Consultas
C16	Consultas por fecha	Consultas
C12	Veterinario por ID	Veterinarios
C5	Consultas por veterinario	Veterinarios
C13	Veterinarios por especialidad	Veterinarios
C22	Veterinarios con más consultas (ranking)	Veterinarios
C11	Servicio por ID	Servicios
C10	Inventario de productos por stock	Productos
C14	Producto por ID	Productos

(La numeración sigue el diagrama original de queries; por eso no hay un «C18».)

12.2 Cómo se arma un reporte

Reportes no tiene base propia: cada reporte combina llamadas HTTP a Animales y/o Consulta. El ejemplo más completo es **C20** («Rescatistas y animales por organización»): pide la organización, luego sus rescatistas, y para CADA rescatista sus animales; el «join» organización → rescatistas → animales se arma **en memoria** en el servicio de Reportes, porque los datos viven en servicios distintos.

- Un reporte útil de rendimiento es el **ranking de veterinarios por cantidad de consultas (C22)**, paginado.

Dónde mirar: `Pawpaws-Reportes/Services/ReporteService.cs` (`C20_OrganizacionDetalleAsync`, `C22_VeterinariosPorConsultasAsync`)

P: ¿Y los datos huérfanos en los reportes?

Se resuelven los nombres a prueba de huérfanos: si una consulta apunta a un veterinario dado de baja se muestra «(dado de baja)», y si ya no existe, «(eliminado)», nunca un GUID suelto.

Dónde mirar: `ResolverNombresVetAsync` / `ResolverNombresAnimalAsync` en `ReporteService.cs`

P: ¿Por qué el listado de consultas no trae servicios ni productos?

Para evitar un problema **N+1** caro en una lista. El listado es liviano; los servicios y productos se cargan solo al abrir el detalle de una consulta.

Dónde mirar: `ObtenerPorCodigoAsync` en `Pawpaws-Consulta/Services/ConsultaService.cs`

13. Ver las queries reales en Cassandra

Cassandra se despliega como un **único clúster** (papaws-cluster) de **dos nodos** en contenedores Docker: papaws-cassandra-animales (puerto 9042) y papaws-cassandra-consulta (puerto 9142). Ambos comparten `cluster_name` y `seeds`, así que forman un solo anillo. Conviven dos keyspaces —`papaws_animales` y `papaws_consulta`— y como ambos usan `replication_factor = 2`, **cada keyspace está replicado en los dos nodos**. La separación entre servicios es **lógica** (cada servicio abre solo su keyspace), no física. Cassandra NO loguea cada query por defecto; para «verlas» se activa **TRACING**.

Implicación de la réplica. Como los dos servicios declaran ambos nodos como *contact points* y cada keyspace tiene una copia en cada nodo (RF = 2), la caída de un nodo no tumba a nadie: el otro nodo responde. Esto se usa en la respuesta sobre el punto único de falla (sección 17).

13.1 Forma A — Una query puntual (cqlsh)

```
docker exec -it papaws-cassandra-animales cqlsh
USE papaws_animales;
TRACING ON;
SELECT * FROM animales_by_rescatista WHERE rescatista_id = <uuid>;
```

13.2 Forma B — Capturar TODAS las queries (la que más sirve)

```
# Activar el tracing al 100 %
docker exec papaws-cassandra-animales nodetool settraceprobability 1

# Usar la app (abrir una ficha, generar un reporte) y luego leer las trazas:
docker exec papaws-cassandra-animales cqlsh -e \
  "SELECT started_at, request, parameters FROM system_traces.sessions LIMIT 50;"

# APAGARLO al terminar (al 100 % es caro):
docker exec papaws-cassandra-animales nodetool settraceprobability 0
```

La columna parameters trae el CQL real.

13.3 Forma C — Inspeccionar datos directos (demostrar la desnormalización)

```
docker exec papaws-cassandra-animales cqlsh -e \
  "SELECT * FROM papaws_animales.animales_by_rescatista LIMIT 20;"

# Ver todas las tablas con su clave de partición y clustering:
docker exec papaws-cassandra-animales cqlsh -e "DESCRIBE KEYSPACE papaws_animales;"
```

13.4 Forma D — Logs

```
docker logs -f papaws-animales # requests HTTP que llegan al microservicio .NET
```

Cruzado con la Forma B muestra el camino completo: request HTTP → servicio → query a Cassandra.

14. Comandos de terminal para la demo

Comandos que el ingeniero puede pedirte en la terminal del proyecto para **mostrar algo en vivo**. Asumen el sistema levantado con Docker. Para ver las *queries* CQL que dispara la app está el **tracing** de la sección 13.

¿DÓNDE escribo estos comandos? Leé esto primero (es el error más común). Hay dos maneras de llegar a Cassandra; **no las mezcles**:

- **Tu terminal de Windows** (PowerShell o CMD, *fuera* de Docker): ahí funcionan los comandos que empiezan con docker... (como docker exec -it papaws-cassandra-animales cqlsh). El programa docker **solo existe en tu terminal del sistema**.
- **La pestaña «Exec» de Docker Desktop**: esa consola **ya está adentro del contenedor**. Ahí **NO** escribas docker exec... (te dirá docker: not found). Escribí directamente cqlsh y listo.

Una vez *dentro* de cqlsh (cursor cqlsh>), los comandos CQL (USE, DESCRIBE, SELECT) son iguales en las dos formas. **Cada uno termina en punto y coma ;** y los keyspaces llevan **guion bajo**: papaws_animales, papaws_consulta (no guion).

14.1 Ver que todo está corriendo

```
docker ps # todos los contenedores y sus puertos
docker compose ps # solo los del proyecto
```

14.2 Demostrar que es un clúster distribuido (2 nodos)

El comando estrella para una defensa de sistemas distribuidos: muestra el anillo con sus dos nodos en estado UN (Up/Normal) y el porcentaje de datos de cada uno.

```
docker exec papaws-cassandra-animales nodetool status
```

Dónde mirar: si ves dos filas UN, son los dos nodos del mismo anillo (papaws-cluster).

14.3 Ver los keyspaces y su factor de réplica

```
docker exec papaws-cassandra-animales cqlsh -e \  
"SELECT keyspace_name, replication FROM system_schema.keyspaces \  
WHERE keyspace_name IN ('papaws_animales', 'papaws_consulta');"
```

Muestra replication_factor = 2: cada keyspace está copiado en los dos nodos.

14.4 Ver las tablas y sus claves (modelado query-first)

```
docker exec papaws-cassandra-animales cqlsh -e "DESCRIBE KEYSPACE papaws_animales;"  
docker exec papaws-cassandra-consulta cqlsh -e "DESCRIBE KEYSPACE papaws_consulta;"
```

En cada CREATE TABLE se ve la PRIMARY KEY: el primer paréntesis es la clave de partición; lo que sigue son las columnas de *clustering*.

14.5 Contar y ver datos de una tabla

```
docker exec papaws-cassandra-consulta cqlsh -e \  
"SELECT COUNT(*) FROM papaws_consulta.servicios_by_id;"  
  
docker exec papaws-cassandra-consulta cqlsh -e \  
"SELECT nombre, precio_base FROM papaws_consulta.servicios_by_id;"
```

14.6 Levantar, reiniciar y ver logs

```
docker compose up --build # levanta Cassandra + los 3 servicios  
docker compose restart papaws-consulta  
docker logs -f papaws-consulta # salida del microservicio .NET
```

14.7 Front en modo desarrollo

```
cd Pawpaws-Front  
npm install  
npm run dev # abre http://localhost:5173
```

14.8 Probar un endpoint por terminal (con token)

Casi todo exige JWT. Primero se hace *login* (devuelve un token) y después se usa en el header Authorization.

```
# 1) Login: copia el valor de "token" de la respuesta  
curl -s -X POST http://localhost:8080/api/auth/login \  
-H "Content-Type: application/json" \  
-d "{\"email\":\"TU_EMAIL\", \"password\":\"TU_PASSWORD\"}"
```

```
# 2) Usar el token en otra llamada (ej. listar servicios):
curl -s http://localhost:8081/api/servicios \
-H "Authorization: Bearer PEGAR_TOKEN_AQUI"
```

Dónde mirar: [login en](#) [Pawpaws-Animales/Controllers/AuthController.cs] ([POST /api/auth/login](#), *campos* email y password).

Truco para la demo. Encadená `nodetool status` (prueba los 2 nodos) + `DESCRIBE KEYSPACE` (prueba el modelado) + el *tracing* de la sección 13 (prueba qué query real se ejecutó). Con esos tres tenés cubierto «distribuido», «modelado» y «en vivo».

14.9 Chuleta: «si te pide esto, escribís esto»

Cada bloque es lo que te puede decir el ingeniero y, debajo, el comando exacto que pegás y lo que va a aparecer. **Todo es copiar y pegar.**

1. «Mostrame que de verdad es distribuido / que hay dos nodos.»

```
docker exec papaws-cassandra-animales nodetool status
```

Aparecen **dos** filas que empiezan con UN (Up/Normal): son los dos nodos del mismo clúster.

2. «Mostrame qué contenedores están corriendo.»

```
docker ps
```

Lista los contenedores (los 3 servicios y las 2 Cassandra) con sus puertos.

3. «Mostrame las tablas del servicio de animales.»

```
docker exec papaws-cassandra-animales cqlsh -e "DESCRIBE KEYSPACE papaws_animales;"
```

Muestra todas las tablas con su PRIMARY KEY.

4. «Mostrame los datos que hay en una tabla.»

```
docker exec papaws-cassandra-animales cqlsh -e \
"SELECT * FROM papaws_animales.animales_by_rescatista LIMIT 10;"
```

Trae hasta 10 filas reales de esa tabla.

5. «¿Cuántos registros hay en esa tabla?»

```
docker exec papaws-cassandra-consulta cqlsh -e \
"SELECT COUNT(*) FROM papaws_consulta.servicios_by_id;"
```

Devuelve un número: la cantidad de filas.

6. «Mostrame que cada base está replicada en los dos nodos.»

```
docker exec papaws-cassandra-animales cqlsh -e \
"SELECT keyspace_name, replication FROM system_schema.keyspaces \
WHERE keyspace_name IN ('papaws_animales', 'papaws_consulta');"

```

En replication se lee 'replication_factor': '2'.

7. «Mostrame la consulta (CQL) que se ejecutó de verdad.»

```
docker exec papaws-cassandra-animales nodetool settraceprobability 1
# (ahora usá la app: abrí una ficha o generá un reporte)
docker exec papaws-cassandra-animales cqlsh -e \
"SELECT started_at, parameters FROM system_traces.sessions LIMIT 20;"
docker exec papaws-cassandra-animales nodetool settraceprobability 0
```

En parameters aparece el texto CQL real. **Acordate de apagarlo** con el último comando.

8. «Mostrame los logs del servicio de consulta.»

```
docker logs --tail 30 papaws-consulta
```

Muestra las últimas 30 líneas de actividad del microservicio .NET.

14.10 Paso a paso: mostrar una consulta en Cassandra (desde DESCRIBE)

Esto es lo más pedido. Seguí los pasos **en orden**; cada uno dice qué escribir y qué vas a ver. (Es para el servicio de *animales*; para *consulta* cambiás animales por consulta en los nombres.)

1. Verificá que Cassandra esté preñada.

```
docker ps
```

Tienen que aparecer papaws-cassandra-animales y papaws-cassandra-consulta. Si no están: docker compose up -d.

2. Entrá a la consola de Cassandra (cqlsh). Según cómo la abras:

```
# A) Desde tu terminal de Windows (PowerShell / CMD):
docker exec -it papaws-cassandra-animales cqlsh

# B) Si estás en la pestaña "Exec" de Docker Desktop (ya estás adentro), solo:
cqlsh
```

En ambos casos el cursor cambia a cqlsh>: ya estás *dentro* de Cassandra. Usá **una** de las dos, no las dos juntas.

3. Entrá al keyspace (la «base» de ese servicio).

```
USE papaws_animales;
```

Ahora el cursor dice cqlsh:papaws_animales>.

4. Mirá qué tablas hay (acá está el DESCRIBE).

```
DESCRIBE TABLES;
```

Aparece la lista: animales_by_id, animales_by_rescatista, rescatistas_by_id, etc.

5. (Opcional) Mirá cómo está hecha una tabla y su clave.

```
DESCRIBE TABLE animales_by_rescatista;
```

Muestra las columnas y la PRIMARY KEY. La primera columna entre paréntesis es la **clave de partición**: el campo por el que se busca rápido.

6. Mostrá los datos de la tabla (esta es «la consulta»).

```
SELECT * FROM animales_by_rescatista LIMIT 10;
```

Aparecen filas reales: rescatista_id, el id del animal, nombre, especie...

7. (Para lucirte) Filtrá por la clave de partición. Copiá un rescatista_id de la salida anterior y pegalo en lugar de PEGA_EL_UUID:

```
SELECT * FROM animales_by_rescatista WHERE rescatista_id = PEGA_EL_UUID;
```

Devuelve solo los animales de ese rescatista, en **una sola lectura** de una partición (sin JOIN). Eso es el modelado query-first.

8. Salí de la consola.

```
exit
```

Si te trabás (errores típicos).

- docker: not found → estás *adentro* del contenedor (pestaña «Exec»): no uses docker..., escribí directamente `cqlsh`.
- Improper DESCRIBE command → a DESCRIBE hay que decirle qué describir y cerrar con `;`: `DESCRIBE TABLES;` o `DESCRIBE KEYSPACES;`.
- Keyspace que no existe → va con **guion bajo** (`papaws_animales`), no con guion (`papaws-animales`).
- Todo comando CQL termina en **punto y coma** `;`.
- El `uuid` del paso 7 va **sin comillas** (es un identificador).
- `ALLOW FILTERING` → estás filtrando por una columna que *no* es la clave de partición: elegí otra tabla o filtrá por su clave (justo lo que demuestra el modelado `query-first`).

15. Partes del código fáciles de cambiar (en vivo)

Si te piden modificar algo en caliente, estos son cambios de pocas líneas y dónde están.

Cómo se aplica cada cambio. **Front** (`Pawpaws-Front`): Vite **recarga solo** al guardar; se ve al instante. **Backend** (.NET): hay que **reconstruir** el servicio (`docker compose up --build <servicio>` o `dotnet run`) para que tome efecto.

15.1 Cupo máximo de animales por rescatista

Cambiar el número (p. ej. de 3 a 5). Hay **dos lugares** que deben coincidir:

- Backend (la regla real): constante `CapacidadMaxima` en `Pawpaws-Animales/Models/Rescatista.cs`.
- Front (solo el texto del aviso): `MAX_POR_RESCATISTA` en `Pawpaws-Front/src/pages/Rescatistas.tsx`.

Dónde mirar: reconstruir Animales tras el cambio; el front es solo cosmético.

15.2 Cuántos elementos por página

El valor por defecto es 10. Se cambia en el hook (afecta a todas las listas) o en una llamada puntual.

- General: `pageSize = 10` en `Pawpaws-Front/src/hooks/usePaginated.ts`.
- Una lista puntual: el 2.º argumento de `usePaginated(lista, 10, ...)` en la página.

Dónde mirar: solo front; se ve al guardar.

15.3 Precios de servicios y productos sembrados

Editar los arreglos `serviciosData` / `productosData` en `Pawpaws-Consulta/Controllers/SeedController.cs`. Cambiar el código **no** reescribe solo lo que ya está en Cassandra: hay que **re-sembrar** o editar el precio puntual desde la pantalla de Servicios/Productos.

15.4 Símbolo y formato de la moneda

Se arma con `Intl.NumberFormat("es-BO", { currency: "BOB" })`. Para cambiar el símbolo, los decimales o el separador de miles (`useGrouping`), se toca ese formateador en las páginas que muestran dinero: `Gastos.tsx`, `Productos.tsx`, `Servicios.tsx` y `ServicioDetalle.tsx`.

15.5 Tamaño de página por defecto del backend

TamanoPorDefecto en `Common/Paginacion.cs` (hay uno por servicio).

15.6 Umbral de «stock bajo»

Un producto se marca «stock bajo» con 10 unidades o menos. Cambiá la comparación `p.stockDisponible <= 10` en `Pawpaws-Front/src/pages/Productos.tsx`.

15.7 Catálogos: especies, especialidades, tipos de organización

- Especies y especialidades: `Pawpaws-Front/src/constants.ts` (`ESPECIES_ACEPTADAS`, `ESPECIALIDADES_VETERINARIAS`).
- Tipos de organización: `TIPOS_ORGANIZACION` en `Pawpaws-Front/src/pages/Organizaciones.tsx`; el backend valida los suyos en `OrganizacionService.cs`.

15.8 Una regla de validación

Las reglas de negocio viven en el *Service* dueño del dato. Por ejemplo, «fecha de ingreso no futura» y «nombre no repetido» se comprueban en `AnimalService.cs`, `ProductoService.cs` y `OrganizacionService.cs`. Cambiar el mensaje o el umbral es editar esa comprobación y reconstruir el servicio.

16. Resumen: quién llama a quién

- **Front** → **Animales**: rescatistas, organizaciones, animales y sus historiales.
- **Front** → **Consulta**: veterinarios, servicios, productos, consultas y gastos.
- **Front** → **Reportes**: todos los reportes.
- **Consulta** → **Animales** (HTTP): solo para validar el animal y su estado al crear una consulta.
- **Reportes** → **Animales** y **Reportes** → **Consulta** (HTTP): para armar cada reporte combinando datos.
- **Ningún servicio entra al keyspace de otro**: siempre por la API REST, con el token JWT reenviado.

17. Preguntas trampa y cómo salir bien

P: ¿Esto es realmente distribuido o son tres apps cualquiera?

Es distribuido: tres servicios desplegados por separado, con **keyspaces independientes**, comunicados por red (HTTP/REST), con fallas y consistencia gestionadas explícitamente (reintentos, validación al escribir, eventos). No comparten datos lógicos ni memoria.

P: ¿No es contradictorio decir «bases independientes» si comparten clúster?

No, hay que distinguir dos planos. A nivel **lógico** cada servicio tiene su propio keyspace y solo abre ese (separación de datos y de dominio real). A nivel **físico** los dos keyspaces viven en el mismo clúster de dos nodos por simplicidad de despliegue; nada impide ponerlos en clústeres separados sin tocar el código, porque la frontera real es el keyspace, no el contenedor.

P: ¿Qué pasa si dos servicios se desincronizan (un id que ya no existe)?

Se asume y se maneja: los reportes resuelven nombres a prueba de huérfanos, y las escrituras críticas validan antes de crear. La consistencia es **eventual** y defendida en cada punto de escritura.

P: ¿Por qué Cassandra y no PostgreSQL?

Por escala de escritura y disponibilidad: Cassandra reparte por particiones y no depende de un único nodo. El costo es modelar *query-first* y desnormalizar, que es justamente lo que mostramos.

P: ¿Dónde está el punto único de falla?

A nivel de aplicación cada servicio y cada keyspace son independientes. A nivel físico, los dos keyspaces viven en el mismo clúster Cassandra de dos nodos con `replication_factor = 2`: por eso la caída de **un** nodo no tumba a nadie (el otro nodo tiene una copia y ambos servicios tienen los dos nodos como *contact points*). Si el *servicio* de Animales cae, Consulta no puede crear consultas nuevas (porque valida el animal) pero el resto sigue; la caída se traduce en un error controlado, no en datos corruptos.

P: ¿Cómo demuestro en vivo que un servicio no toca el keyspace de otro?

Mostrando que Reportes no tiene cadena de conexión a ninguna base (solo clientes HTTP) y que cada keyspace solo lo abre su propio servicio (`ConnectAsync(keyspace)` con su propio keyspace en cada `Program.cs`). Con la Forma B del tracing se ve que las queries del keyspace `papaws_consulta` solo las origina el servicio de Consulta.

18. Banco de preguntas rápidas

Respuestas cortas para repasar de memoria y para preguntas sueltas.

18.1 Arquitectura y comunicación

P: ¿Qué protocolo usan los servicios entre sí?

HTTP/REST con JSON.

P: ¿La comunicación es síncrona o asíncrona?

Síncrona (*request/response*). No hay colas ni mensajería: las llamadas cruzadas son validaciones puntuales al escribir.

P: ¿Cómo encuentra un servicio a otro?

Por URL configurada (en Docker, por el nombre del contenedor). No hay *service discovery* dinámico.

P: ¿Qué pasa si cambio el puerto de un servicio?

Se ajusta en `docker-compose.yml` y en las URLs del front (su `.env`).

P: ¿Cuántos reportes hay?

Más de veinte (C1...C22): de rescatistas, animales, consultas, veterinarios, servicios y productos.

18.2 Cassandra

P: ¿Clave de partición vs clustering?

Partición = en qué nodo vive y por qué campo se busca rápido; clustering = el orden de las filas dentro de esa partición.

P: ¿Por qué no usar ALLOW FILTERING?

Porque hace *scan* y no escala; en su lugar se crea una tabla (o índice) por cada consulta de lectura.

P: ¿Qué pasa si agrego una columna a una tabla?

`ALTER TABLE` en caliente; las filas viejas la tienen en null (justo lo que pasó con `costo_unitario` antes de cargar precios).

P: ¿Por qué 2 nodos con RF = 2?

Para tolerar la caída de un nodo: cada dato tiene una copia en ambos.

P: ¿Cómo evitan inyección en las queries?

Con *prepared statements* parametrizados (no se concatena texto); además se compilan una vez y se reutilizan.

18.3 .NET y front

P: ¿Cómo se conecta el backend a Cassandra?

Con el driver de DataStax: una sesión por keyspace, abierta al arrancar en Program.cs.

P: ¿Dónde se registran los servicios?

Inyección de dependencias en el Program.cs de cada microservicio.

P: ¿Dónde guarda el front el token?

En el navegador, y lo manda en el header Authorization en cada llamada.

Dónde mirar: Pawpaws-Front/src/auth/AuthContext.tsx y `authHeaders()` en src/api/client.ts

P: ¿Qué pasa cuando el token expira?

El backend responde 401 y el front lo maneja cerrando sesión / volviendo al login.

18.4 Negocio

P: ¿Qué roles existen?

Administrador, Encargado de consultas y Encargado de rescatistas; cada endpoint exige el rol que corresponde.

P: ¿Un animal puede tener varias consultas?

Sí; la relación es 1-N por animal_id.

P: ¿Qué pasa con el stock al cancelar una consulta?

Se devuelve, y por eso esa consulta no cuenta como gasto.

P: ¿Un producto vencido se borra?

No; se marca su vencimiento y el front lo resalta. La baja siempre es lógica.

Guía de defensa del proyecto Papaws.

Complementaria a «Funcionamiento del proyecto Papaws» y «Esquema de tablas en Cassandra».